

Software Testing Methodologies Lab

Experiment 1 - Recording in Context Sensitive Mode and Analog Mode (Selenium IDE)

Tools/Requirements:

- Selenium IDE extension (Chrome/Edge)
- Any demo site for drag-and-drop (optional)

Procedure:

1. Open Chrome/Edge and install the Selenium IDE extension from the browser extensions store.
2. Click the Extensions icon and open Selenium IDE.
3. Click 'Create a new project' and enter a project name.
4. Click 'Add new test' and name the test case (e.g., TC_Record_Google).
5. Click the red 'Record' button.
6. Enter the base URL (e.g., <https://www.google.com>) and click 'Start Recording'.
7. Perform actions: click the search box, type a keyword, press Enter, click a result, then go back.
8. Stop recording and review the generated commands (open, click, type, assert).
9. Run the test using the 'Run current test' (play) button.
10. For analog-style actions: try recording a drag-and-drop or slider movement on a demo site and replay it.
11. Save the project in Selenium IDE.

Notes:

- Selenium IDE is mainly context-sensitive (element based). True analog recording is limited.
- For complex gestures use Selenium WebDriver code with the Actions class.

Output:

```
Running 'TC_Record_Google'  
open | https://www.google.com |  
type | name=q | selenium  
assertTitle | Google |  
Test completed successfully  
Result: PASS
```

Software Testing Methodologies Lab

Experiment 2 - GUI Checkpoint for Single Property

Java Code

```
import javax.swing.JFrame;

public class Exp2_SinglePropertyCheckpoint {
    public static void main(String[] args) throws InterruptedException {
        JFrame frame = new JFrame("Google");
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        String expectedTitle = "Google";
        String actualTitle = frame.getTitle();

        System.out.println("Actual Title : " + actualTitle);
        System.out.println("Expected Title : " + expectedTitle);

        if (actualTitle.equals(expectedTitle)) {
            System.out.println("PASS: Title is correct");
        } else {
            System.out.println("FAIL: Title is incorrect");
        }

        Thread.sleep(2000);
        frame.dispose();
    }
}
```

Compile / Run Commands

```
javac Exp2_SinglePropertyCheckpoint.java
java Exp2_SinglePropertyCheckpoint
```

Output

```
Actual Title : Google
Expected Title : Google
PASS: Title is correct
```

Experiment 3 - GUI Checkpoint for Single Object

Java Code

```
import javax.swing.*;

public class Exp3_SingleObjectCheckpoint {
    public static void main(String[] args) throws InterruptedException {
        JFrame frame = new JFrame("Single Object Checkpoint");
        JTextField searchBox = new JTextField(20);

        JPanel panel = new JPanel();
        panel.add(new JLabel("Search:"));
        panel.add(searchBox);

        frame.add(panel);
        frame.setSize(400, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        Thread.sleep(500);

        boolean displayed = searchBox.isShowing();
        boolean enabled = searchBox.isEnabled();

        System.out.println("Displayed: " + displayed);
        System.out.println("Enabled : " + enabled);
    }
}
```

```

        System.out.println(displayed && enabled
            ? "PASS: Search box is ready"
            : "FAIL: Search box is not ready");

        Thread.sleep(2000);
        frame.dispose();
    }
}

```

Compile / Run Commands

```

javac Exp3_SingleObjectCheckpoint.java
java Exp3_SingleObjectCheckpoint

```

Output

```

Displayed: true
Enabled : true
PASS: Search box is ready

```

Experiment 4(a) - Bitmap Checkpoint for a Single Object

Java Code

```

import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

public class NewTest {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Checkpoint Test");
        JTextField searchBox = new JTextField("Search here", 20);
        JPanel panel = new JPanel();
        panel.add(searchBox);
        frame.add(panel);
        frame.setSize(350, 120);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        Thread.sleep(1000);

        File folder = new File("Screenshots");
        if (!folder.exists()) folder.mkdir();

        BufferedImage currentImage = new BufferedImage(
            searchBox.getWidth(), searchBox.getHeight(),
            BufferedImage.TYPE_INT_RGB);
        Graphics2D g2 = currentImage.createGraphics();
        searchBox.paint(g2);
        g2.dispose();

        File currentFile = new File("Screenshots/current_searchbox.png");
        ImageIO.write(currentImage, "png", currentFile);
        System.out.println("Current screenshot saved:");
        System.out.println(currentFile.getAbsolutePath());

        File baselineFile = new File("Screenshots/baseline_searchbox.png");
        if (!baselineFile.exists()) {
            System.out.println("Baseline image not found.");
            System.out.println("Rename current_searchbox.png as baseline_searchbox.png");
        } else {
            boolean result = compareImages(baselineFile, currentFile);
            if (result) {
                System.out.println("BITMAP CHECKPOINT : TEST PASS");
                System.out.println("Images are identical.");
            } else {
                System.out.println("BITMAP CHECKPOINT : TEST FAIL");
                System.out.println("Images are different.");
            }
        }
        Thread.sleep(2000);
        frame.dispose();
    }
}

```

```

    }

    public static boolean compareImages(File img1, File img2) throws Exception {
        BufferedImage image1 = ImageIO.read(img1);
        BufferedImage image2 = ImageIO.read(img2);
        if (image1.getWidth() != image2.getWidth() ||
            image1.getHeight() != image2.getHeight()) return false;

        for (int x = 0; x < image1.getWidth(); x++) {
            for (int y = 0; y < image1.getHeight(); y++) {
                if (image1.getRGB(x, y) != image2.getRGB(x, y)) return false;
            }
        }
        return true;
    }
}

```

Compile / Run Commands

```

javac NewTest.java
java NewTest

```

Sample Output

```

Current screenshot saved:
.../Screenshots/current_searchbox.png
BITMAP CHECKPOINT : TEST PASS
Images are identical.

```



Figure: Sample output image - current_searchbox.png

Note: On the first run, rename current_searchbox.png to baseline_searchbox.png, then run again for comparison.

Experiment 4(b) - Bitmap Checkpoint for a Screen Area

Java Code

```

import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

public class NewTest {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Area Checkpoint");
        JPanel panel = new JPanel(new FlowLayout());
        panel.add(new JLabel("Google Search"));
        panel.add(new JTextField(25));
        panel.add(new JButton("Search"));
        frame.add(panel);
        frame.setSize(800, 600);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        Thread.sleep(1000);

        BufferedImage fullImage = new BufferedImage(
            frame.getWidth(), frame.getHeight(), BufferedImage.TYPE_INT_RGB);
        Graphics2D g2 = fullImage.createGraphics();
        frame.paint(g2);
        g2.dispose();

        File folder = new File("SeleniumScreenshots");
        if (!folder.exists()) folder.mkdirs();

        int width = Math.min(800, fullImage.getWidth());
        int height = Math.min(600, fullImage.getHeight());
        BufferedImage currentArea = fullImage.getSubimage(0, 0, width, height);
    }
}

```

```

File currentFile = new File("SeleniumScreenshots/current_google_area.png");
ImageIO.write(currentArea, "png", currentFile);
System.out.println("Current screenshot saved:");
System.out.println(currentFile.getAbsolutePath());

File baselineFile = new File("SeleniumScreenshots/baseline_google_area.png");
if (!baselineFile.exists()) {
    System.out.println("Baseline image not found.");
    System.out.println("Rename current_google_area.png to baseline_google_area.png");
} else {
    BufferedImage baseline = ImageIO.read(baselineFile);
    double similarity = compareImages(baseline, currentArea);
    System.out.printf("Image Similarity : %.2f%%\n", similarity);
    if (similarity >= 95)
        System.out.println("BITMAP CHECKPOINT : TEST PASS");
    else
        System.out.println("BITMAP CHECKPOINT : TEST FAIL");
}
Thread.sleep(2000);
frame.dispose();
}

public static double compareImages(BufferedImage img1, BufferedImage img2) {
    if (img1.getWidth() != img2.getWidth() ||
        img1.getHeight() != img2.getHeight()) return 0;

    long totalPixels = (long) img1.getWidth() * img1.getHeight();
    long samePixels = 0;
    for (int x = 0; x < img1.getWidth(); x++) {
        for (int y = 0; y < img1.getHeight(); y++) {
            if (img1.getRGB(x, y) == img2.getRGB(x, y)) samePixels++;
        }
    }
    return ((double) samePixels / totalPixels) * 100;
}
}

```

Compile / Run Commands

```

javac NewTest.java
java NewTest

```

Sample Output

```

Current screenshot saved:
.../SeleniumScreenshots/current_google_area.png
Image Similarity : 100.00%
BITMAP CHECKPOINT : TEST PASS

```

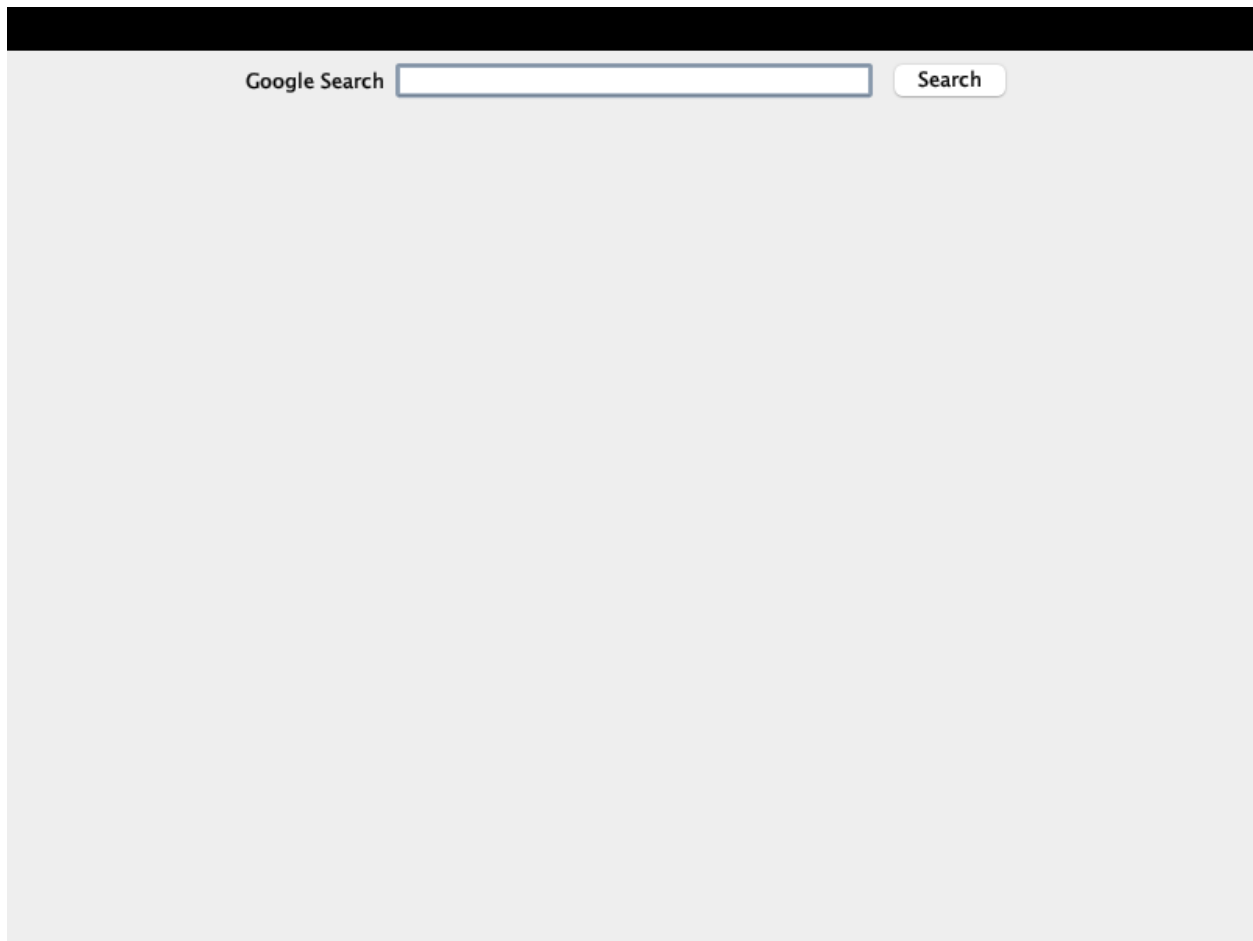


Figure: Sample output image - current_google_area.png

Note: On the first run, rename current_google_area.png to baseline_google_area.png, then run again. PASS is reported when similarity is at least 95%.

Experiment 5 - Database Checkpoint - Default Check (Record Exists)

Java Code

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class NewTest {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:h2:./StudentDB", "sa", "");
            Statement stmt = con.createStatement();

            stmt.execute("CREATE TABLE IF NOT EXISTS STUDENT (" +
                "ID INT PRIMARY KEY, " +
                "NAME VARCHAR(50), " +
                "EMAIL VARCHAR(100))");
            stmt.execute("DELETE FROM STUDENT");
            stmt.executeUpdate("INSERT INTO STUDENT VALUES " +
                "(1, 'Ajay', 'ajay@gmail.com')");

            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM STUDENT WHERE ID = 1");

            if (rs.next()) {
                System.out.println("Record Exists");
            }
        }
    }
}
```

```

        System.out.println("ID : " + rs.getInt("ID"));
        System.out.println("Name : " + rs.getString("NAME"));
        System.out.println("Email : " + rs.getString("EMAIL"));
        System.out.println("TEST PASSED");
    } else {
        System.out.println("Record Not Found");
        System.out.println("TEST FAILED");
    }

    rs.close();
    stmt.close();
    con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Compile / Run Commands

```

javac -cp h2-2.4.240.jar NewTest.java
java -cp ".;h2-2.4.240.jar" NewTest

```

Output

```

Record Exists
ID : 1
Name : Ajay
Email : ajay@gmail.com
TEST PASSED

```

Experiment 6 - Database Checkpoint for Custom Check

Java Code

```

import java.sql.*;

public class DatabaseCheckpointCustomCheck {
    static final String DB_URL = "jdbc:sqlite:student.db";

    public static void setupDatabase() throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             Statement stmt = conn.createStatement()) {
            stmt.execute("CREATE TABLE IF NOT EXISTS students (" +
                "id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)");
            stmt.execute("DELETE FROM students");
            stmt.execute("INSERT INTO students VALUES (1, 'Amit', 78)");
            stmt.execute("INSERT INTO students VALUES (2, 'Neha', 45)");
            stmt.execute("INSERT INTO students VALUES (3, 'Ravi', 92)");
        }
    }

    public static boolean customCheckpoint(int studentId,
                                           String expectedStatus)
        throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement ps = conn.prepareStatement(
                "SELECT marks FROM students WHERE id = ?")) {
            ps.setInt(1, studentId);
            ResultSet rs = ps.executeQuery();

            if (!rs.next()) {
                System.out.println("FAIL: No record found for id " + studentId);
                return false;
            }

            int marks = rs.getInt("marks");
            String actualStatus = (marks >= 50) ? "Pass" : "Fail";

```

```

        if (actualStatus.equals(expectedStatus)) {
            System.out.println("PASS: Student " + studentId +
                " status = " + actualStatus +
                " (Custom check matched)");
            return true;
        } else {
            System.out.println("FAIL: Student " + studentId +
                " expected " + expectedStatus +
                ", got " + actualStatus);
            return false;
        }
    }
}

public static void main(String[] args) throws Exception {
    setupDatabase();
    customCheckpoint(1, "Pass");
    customCheckpoint(2, "Fail");
    customCheckpoint(3, "Pass");
}
}

```

Compile / Run Commands

```

javac -cp sqlite-jdbc-3.34.0.jar DatabaseCheckpointCustomCheck.java
java -cp ".;sqlite-jdbc-3.34.0.jar" DatabaseCheckpointCustomCheck

```

Output

```

PASS: Student 1 status = Pass (Custom check matched)
PASS: Student 2 status = Fail (Custom check matched)
PASS: Student 3 status = Pass (Custom check matched)

```

Experiment 7(a) - Dynamic Test Data Submission

Java Code

```

import java.util.Random;

public class DynamicDataTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void main(String[] args) {
        String[] categories = {"electronics", "clothing", "grocery"};
        Random rand = new Random();
        System.out.println("---- Dynamic Test Data Submission ----");

        for (int i = 1; i <= 5; i++) {
            int price = 100 + rand.nextInt(901);
            String category = categories[rand.nextInt(categories.length)];
            double result = calculateDiscount(price, category);
            System.out.printf(
                "Test %d: price=%d, category=%s -> discounted=%.2f%n",
                i, price, category, result);
        }
    }
}

```

Compile / Run Commands

```

javac DynamicDataTest.java
java DynamicDataTest

```

Output

```

---- Dynamic Test Data Submission ----
Test 1: price=527, category=electronics -> discounted=474.30
Test 2: price=304, category=clothing -> discounted=243.20

```

```
Test 3: price=812, category=grocery -> discounted=812.00
Test 4: price=450, category=electronics -> discounted=405.00
Test 5: price=265, category=clothing -> discounted=212.00
```

Experiment 7(b) - Data Driven Test Through Flat Files

Java Code

```
import java.io.*;

public class FlatFileTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createFlatFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("testdata.txt"))) {
            pw.println("500,electronics");
            pw.println("300,clothing");
            pw.println("150,grocery");
        }
    }

    public static void flatFileTest() throws IOException {
        System.out.println("---- Data Driven Test via Flat File ----");
        try (BufferedReader br = new BufferedReader(
            new FileReader("testdata.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                double price = Double.parseDouble(parts[0]);
                String category = parts[1];
                double result = calculateDiscount(price, category);
                System.out.printf(
                    "price=%.1f, category=%s -> discounted=%.2f%n",
                    price, category, result);
            }
        }
    }

    public static void main(String[] args) throws IOException {
        createFlatFile();
        flatFileTest();
    }
}
```

Compile / Run Commands

```
javac FlatFileTest.java
java FlatFileTest
```

Output

```
---- Data Driven Test via Flat File ----
price=500.0, category=electronics -> discounted=450.00
price=300.0, category=clothing -> discounted=240.00
price=150.0, category=grocery -> discounted=150.00
```

Experiment 7(c) - Data Driven Test Through Front-End Grids

Java Code

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class FrontEndGridTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
```

```

        case "electronics": return price * 0.9;
        case "clothing": return price * 0.8;
        default: return price;
    }
}

public static void main(String[] args) {
    JFrame frame = new JFrame("Front End Grid - Data Driven Test");
    String[] columns = {"Price", "Category", "Result"};
    DefaultTableModel model = new DefaultTableModel(columns, 0);

    Object[][] gridData = {
        {500.0, "electronics"},
        {300.0, "clothing"},
        {150.0, "grocery"}
    };

    for (Object[] row : gridData) {
        double price = (double) row[0];
        String category = (String) row[1];
        double result = calculateDiscount(price, category);
        model.addRow(new Object[]{
            price, category, String.format("%.2f", result)
        });
    }

    JTable table = new JTable(model);
    frame.add(new JScrollPane(table));
    frame.setSize(400, 200);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```

Compile / Run Commands

```

javac FrontEndGridTest.java
java FrontEndGridTest

```

Output

Price	Category	Result
500.0	electronics	450.00
300.0	clothing	240.00
150.0	grocery	150.00

Experiment 7(d) - Data Driven Test Through Excel

Java Code

```

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.*;

public class ExcelDrivenTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createExcelFile() throws IOException {
        Workbook wb = new XSSFWorkbook();
        Sheet sheet = wb.createSheet("TestData");

        Object[][] data = {
            {"Price", "Category"},
            {500.0, "electronics"},
            {300.0, "clothing"},

```

```

        {150.0, "grocery"}
    };

    for (int r = 0; r < data.length; r++) {
        Row row = sheet.createRow(r);
        for (int c = 0; c < data[r].length; c++) {
            Cell cell = row.createCell(c);
            if (data[r][c] instanceof Double)
                cell.setCellValue((Double) data[r][c]);
            else
                cell.setCellValue((String) data[r][c]);
        }
    }

    try (FileOutputStream fos = new FileOutputStream("testdata.xlsx")) {
        wb.write(fos);
    }
    wb.close();
}

public static void excelDrivenTest() throws IOException {
    System.out.println("---- Data Driven Test via Excel ----");
    try (FileInputStream fis = new FileInputStream("testdata.xlsx")) {
        Workbook wb = new XSSFWorkbook(fis) {
            Sheet sheet = wb.getSheetAt(0);

            for (Row row : sheet) {
                if (row.getRowNum() == 0) continue;
                double price = row.getCell(0).getNumericCellValue();
                String category = row.getCell(1).getStringCellValue();
                double result = calculateDiscount(price, category);

                System.out.printf(
                    "price=%.1f, category=%s -> discounted=%.2f%n",
                    price, category, result);
            }
        }
    }

    public static void main(String[] args) throws IOException {
        createExcelFile();
        excelDrivenTest();
    }
}

```

Compile / Run Commands

```

javac -cp ".;poi-5.2.5.jar;poi-ooxml-5.2.5.jar;poi-ooxml-lite-5.2.5.jar;xmlbeans-5.2.0.jar;commons-
collections4-4.4.jar;commons-compress-1.26.0.jar;commons-io-2.15.1.jar;log4j-api-2.20.0.jar"
ExcelDrivenTest.java
java -cp ".;poi-5.2.5.jar;poi-ooxml-5.2.5.jar;poi-ooxml-lite-5.2.5.jar;xmlbeans-5.2.0.jar;commons-
collections4-4.4.jar;commons-compress-1.26.0.jar;commons-io-2.15.1.jar;log4j-api-2.20.0.jar"
ExcelDrivenTest

```

Output

```

---- Data Driven Test via Excel ----
price=500.0, category=electronics -> discounted=450.00
price=300.0, category=clothing -> discounted=240.00
price=150.0, category=grocery -> discounted=150.00

```

Experiment 7(e) - Batch Testing Without Parameter Passing

Java Code

```

public class BatchNoParams {
    public static void testAddition() {
        assert 2 + 3 == 5;
        System.out.println("test_addition: PASS");
    }
}

```

```

    public static void testSubtraction() {
        assert 5 - 2 == 3;
        System.out.println("test_subtraction: PASS");
    }

    public static void main(String[] args) {
        System.out.println("---- Batch Testing (No Parameters) ----");
        testAddition();
        testSubtraction();
    }
}

```

Compile / Run Commands

```

javac BatchNoParams.java
java -ea BatchNoParams

```

Output

```

---- Batch Testing (No Parameters) ----
test_addition: PASS
test_subtraction: PASS

```

Experiment 7(f) - Batch Testing With Parameter Passing

Java Code

```

public class BatchWithParams {
    static void testOperation(int a, int b, char op, int expected) {
        int result = 0;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
        }

        String status = (result == expected) ? "PASS" : "FAIL";
        System.out.printf(
            "Test: %d %c %d = %d (expected %d) -> %s\n",
            a, op, b, result, expected, status);
    }

    public static void main(String[] args) {
        int[][] testCases = {
            {2, 3, '+', 5},
            {10, 4, '-', 6},
            {3, 3, '*', 9},
            {5, 5, '-', 1}
        };

        System.out.println("---- Batch Testing (With Parameters) ----");
        for (int[] tc : testCases) {
            testOperation(tc[0], tc[1], (char) tc[2], tc[3]);
        }
    }
}

```

Compile / Run Commands

```

javac BatchWithParams.java
java BatchWithParams

```

Output

```

---- Batch Testing (With Parameters) ----
Test: 2 + 3 = 5 (expected 5) -> PASS
Test: 10 - 4 = 6 (expected 6) -> PASS
Test: 3 * 3 = 9 (expected 9) -> PASS
Test: 5 - 5 = 0 (expected 1) -> FAIL

```

Experiment 8 - Data Driven Batch

Java Code

```
import java.io.*;

public class DataDrivenBatch {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createCsvFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("batch_data.csv"))) {
            pw.println("Price,Category,ExpectedResult");
            pw.println("500,electronics,450");
            pw.println("300,clothing,240");
            pw.println("150,grocery,150");
            pw.println("1000,electronics,900");
        }
    }

    public static void dataDrivenBatch() throws IOException {
        System.out.println("---- Data Driven Batch Test ----");
        int passCount = 0, failCount = 0;

        try (BufferedReader br =
            new BufferedReader(new FileReader("batch_data.csv"))) {
            String line = br.readLine();

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                double price = Double.parseDouble(parts[0]);
                String category = parts[1];
                double expected = Double.parseDouble(parts[2]);
                double actual = calculateDiscount(price, category);

                if (actual == expected) {
                    System.out.printf(
                        "PASS: %s, price=%.1f -> %.1f\n",
                        category, price, actual);
                    passCount++;
                } else {
                    System.out.printf(
                        "FAIL: %s, price=%.1f -> got %.1f, expected %.1f\n",
                        category, price, actual, expected);
                    failCount++;
                }
            }
        }

        System.out.println("\nBatch Summary: " + passCount +
            " Passed, " + failCount + " Failed");
    }

    public static void main(String[] args) throws IOException {
        createCsvFile();
        dataDrivenBatch();
    }
}
```

Compile / Run Commands

```
javac DataDrivenBatch.java
java DataDrivenBatch
```

Output

```
---- Data Driven Batch Test ----  
PASS: electronics, price=500.0 -> 450.0  
PASS: clothing, price=300.0 -> 240.0  
PASS: grocery, price=150.0 -> 150.0  
PASS: electronics, price=1000.0 -> 900.0
```

Batch Summary: 4 Passed, 0 Failed

Experiment 9 - Silent Mode Test Execution Without Interruption

Java Code

```
import java.util.logging.*;  
import java.io.IOException;  
  
public class SilentModeTest {  
    static Logger logger = Logger.getLogger("SilentTestLogger");  
  
    static void setupLogger() throws IOException {  
        logger.setUseParentHandlers(false);  
        FileHandler fh = new FileHandler("silent_test_log.txt");  
        fh.setFormatter(new SimpleFormatter());  
        logger.addHandler(fh);  
    }  
  
    static void testCase(String name, Object actual, Object expected) {  
        try {  
            if (actual.equals(expected)) {  
                logger.info(name + ": PASS (actual=" + actual +  
                    ", expected=" + expected + ")");  
            } else {  
                logger.severe(name + ": FAIL (actual=" + actual +  
                    ", expected=" + expected + ")");  
            }  
        } catch (Exception e) {  
            logger.severe(name + ": ERROR - " + e.getMessage());  
        }  
    }  
  
    public static void main(String[] args) throws IOException {  
        setupLogger();  
        testCase("Addition Test", 2 + 2, 4);  
        testCase("Subtraction Test", 10 - 5, 5);  
        testCase("Multiplication Test", 3 * 3, 10);  
        testCase("String Test", "abc".toUpperCase(), "ABC");  
  
        System.out.println(  
            "Batch execution completed silently. " +  
            "Check silent_test_log.txt for results.");  
    }  
}
```

Compile / Run Commands

```
javac SilentModeTest.java  
java SilentModeTest  
type silent_test_log.txt
```

Output

Terminal:
Batch execution completed silently. Check silent_test_log.txt for results.

Log file (message portion):
INFO: Addition Test: PASS (actual=4, expected=4)
INFO: Subtraction Test: PASS (actual=5, expected=5)

SEVERE: Multiplication Test: FAIL (actual=9, expected=10)

INFO: String Test: PASS (actual=ABC, expected=ABC)

Experiment 10 - Test Case for Calculator in Windows Application

Java Code

```
import java.awt.*;
import java.awt.event.KeyEvent;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;

public class CalculatorTest {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().exec("calc.exe");
        Thread.sleep(2000);

        Robot robot = new Robot();
        robot.setAutoDelay(200);

        typeKey(robot, KeyEvent.VK_7);
        typeKey(robot, KeyEvent.VK_ADD);
        typeKey(robot, KeyEvent.VK_5);
        typeKey(robot, KeyEvent.VK_EQUALS);

        Thread.sleep(1000);

        Rectangle screenRect = new Rectangle(
            Toolkit.getDefaultToolkit().getScreenSize());

        BufferedImage capture =
            robot.createScreenCapture(screenRect);

        ImageIO.write(capture, "png",
            new File("calculator_result.png"));

        System.out.println("Test executed: 7 + 5 =");
        System.out.println(
            "Screenshot saved as calculator_result.png -- " +
            "compare visually to confirm result is 12");
    }

    static void typeKey(Robot robot, int keyCode) {
        robot.keyPress(keyCode);
        robot.keyRelease(keyCode);
    }
}
```

Compile / Run Commands

```
javac CalculatorTest.java
java CalculatorTest
```

Output

Test executed: 7 + 5 =12

Screenshot saved as calculator_result.png -- compare visually to confirm result is 12

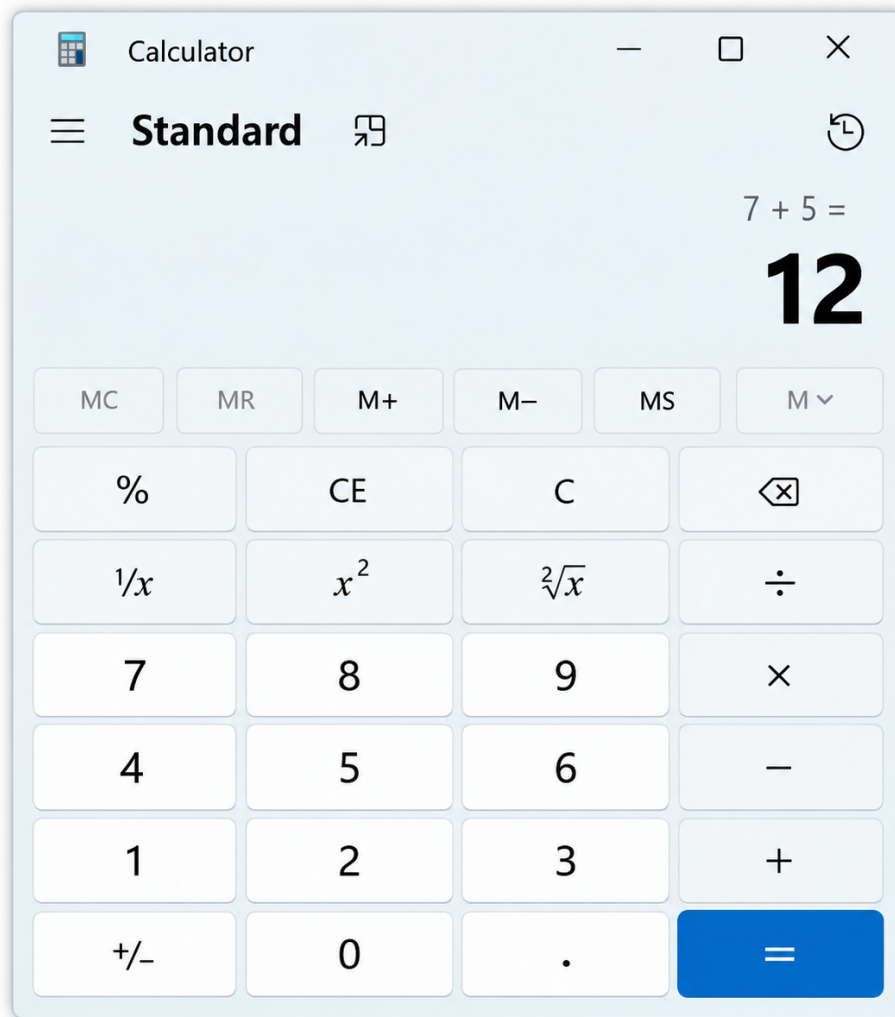


Figure: Illustrative expected calculator result for Experiment 10 ($7 + 5 = 12$)